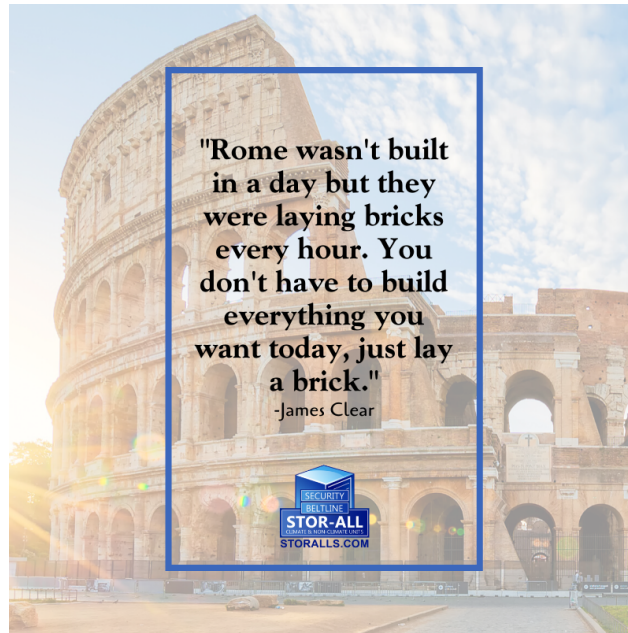


NOTES:

HASHMAP



Good
Morning

Note:- Concept will be correct
Syntax → will share a doc

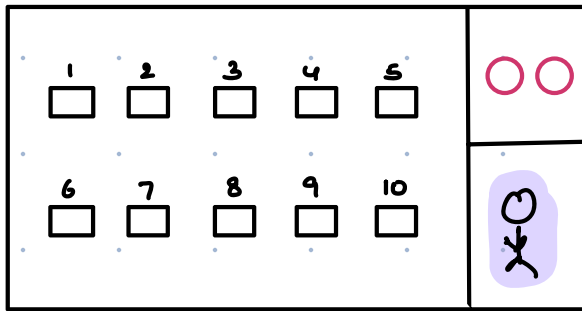
Today's Agenda

- Hashmap Introduction
- HashSet Introduction
- Frequency of each query
- First non repeating ele
- distinct elements
- Subarray with sum = 0 (Interview)

* How things are implemented in HashMap

HashMap implementation

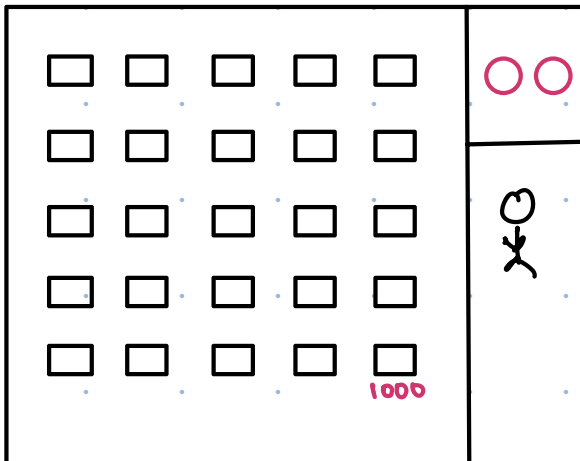
Yogi



Register

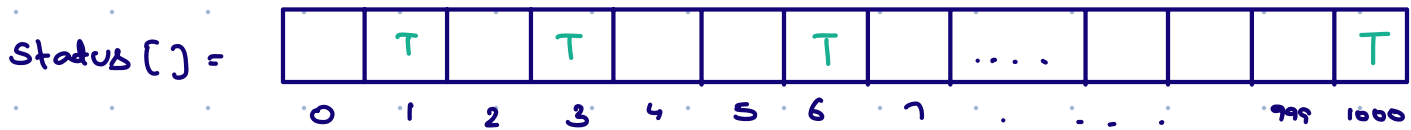
- 1 → Occupied
- 2 → Not occupied
- 3 → Occupied
- ...
- 10 → Occupied

1000 rooms



Room no. → {1, 1000}

boolean array [] = 1001



status array

True → Room is occupied

False → Room is available

To check if room is available or not?

Tc: $O(1)$

* Covid $\rightarrow$ Hotel Business is not doing great.

Numerologist $\rightarrow [1 \text{ to } 10^9]$

1000 rooms

1	7	200	10^4	36	○○
☐	☐	☐	☐	☐	
8	9				O ↓
☐	☐	☐	☐	☐	
☐	☐	☐	☐	☐	
☐	☐	☐	☐	☐	
☐	☐	☐	☐	☐	
				1000	

One on one mapping

boolean [] status = $10^9 + 1$

Advantage $\rightarrow$ Access the room status $\rightarrow$ Tc: $O(1)$

Issue

$\rightarrow$ For 1000 rooms

$\hookrightarrow$ space of $10^9 + 1$

consuming a lot of extra
space to store information
of 1000 rooms

HashMap $\rightarrow$ A DS which holds the information in
key-value pairs

HashMap<key, value>

Room no. T/F

$\left\{ \begin{array}{l} 10^4 \rightarrow \text{True} \\ 8 \rightarrow \text{True} \\ 9 \rightarrow \text{True} \end{array} \right\}$

max Size(hm) = 1000

any room

hm[9] = True

room 9 is
occupied

check the value for
a particular room $\rightarrow$ TC: $O(1)$

* Implementation of HM & its working $\downarrow$
covered later in
classes

* Keys $\rightarrow$ Always be unique
Value $\rightarrow$ Can be anything

o. Store population of every country

key : Country name {string}

value : Population {long}

HM <string, long>

02. No. of states for each country

key : Country name {String}

Value : No. of states {Integer}

HM < String, Integer >

03. Name of all states of each country

key : Country name {String}

Value : Name of { ArrayList <String> }
all states

HM < String, ArrayList <String> >

04. Store population of each state in every country

key : Country name {String}

Value : population of each state : { HashMap <String, Long> }

HM < String, HashMap <String, Long> >

HashMap → To store key-value pairs

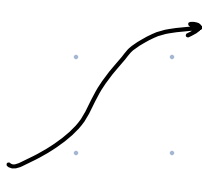
HashSet → To store unique keys

HashMap	HashSet
01. size () → No. of keys in HM 02. insert (key, value) 03. search (key) → T/F 04. remove (key) 05. update → insert (key, value) 06. get (key) → Fetch the value for this key 07. iterate on hm ↳ get all the keys	01. size () → No. of keys in hashset 02. add (key) 03. search (key) → T/F 04. remove (key) 05. Iterate on hashset ↓ to get all keys

A single operation in HashMap / HashSet is

TC: $O(1)$

* India → 20
 * USA → 23
 * Pak → 21
 * Swi → 17
 * France → 4
 * India → 4



```

HashMap<String, Integer>
  hm.insert (India, 20);
  hm.insert (USA, 23);
  hm.insert (Pak, 21);
  hm.insert (Swi, 17);
  hm.insert (France, 3);
  
```

hm.insert (India, 4);

H M

USA	→	23
Swi.	→	17
India	→	4
Pak	→	21
France	→	4

TC: $O(n)$
SC: $O(n)$ } To insert n entries

- * Ordering inside hashmap will not be same as ordering of insertion.
- * If two duplicate keys are inserted, then there will be only one key, with the modified value. (Overwrite the previous value)

7:56 AM → 8:06 AM

Hashing Library Names in Different Languages

Java

C++

Python

Js

C#

HashMap HashMap unordered_map dictionary map dictionary

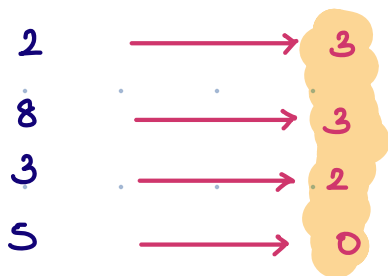
HashSet HashSet unordered_set set set HashSet

Q → Find frequency of numbers

Given N array elements & Q queries, find frequency of elements (given in queries) from the array.

arr[10] = {2 6 3 8 2 8 2 3 8 10}

Queries: 4



Brute force → For every query, iterate on the array & count its frequency.

Tc: $O(Q * n)$

Sc: $O(1)$

02. Sort the array

Tc: $O(n \log n + Q * n)$

Sc: $O(1)$

03. Optimised Approach = Use Hashmap

Key → Distinct ele

Value → Freq of distinct ele

arr[10] = {2 6 3 8 2 8 2 3 8 10}

hm < Integer, Integer > map

key	value
2	→ 1 2 3
6	→ 1
3	→ 1 2
8	→ 1 2 3
10	→ 1

Queries

2 → map.get(2)
8 → map.get(8)
3 → map.get(3)
5 → map.get(5)

void printquery (int [] A, int [] Q)

HashMap < Integer, Integer > hm;

// Iterate on array & populate hm

for (i=0; i<n; i++) {

if (hm.search(A[i]) == True) {

int of = hm.get(A[i]);

int nf = of + 1

hm.insert(A[i], nf)

else {

hm.insert(A[i], 1);

// Iterate on queries

TC: $O(Q+n)$

SC: $O(n)$

```
for (i=0; i < Q.length; i++) {
```

```
    int ele = Q[i];
```

```
    if (hm.search(ele) == True) {
```

```
        | print(hm.get(ele));
```

```
    }
```

```
    else print(0);
```

```
}
```

}

02 Find the first non repeating ele.

↳ first ele which is not repeating from start

$A[] = \{4, 3, 3, 2, 5, 6, 4, 5\}$ Ans = 2

$A[] = \{1, 3, 2, 1, 4\}$ Ans = 3

$A[] = \{2, 3, 4, 1\}$ Ans = 2

BF → For every element, iterate & look for its frequency. If freq == 1 → return ans.

$Tc: O(n^2)$

$Sc: O(1)$

* Optimal Approach $\rightarrow$ To use hashmap

01. Build hashmap.

02. Iterate on array & return first ele with freq as 1

$\rightarrow$
 $A[] = \{ 1, 4, 3, 6, 3, 4, 1, 7, 3 \}$

HM =

key	value
7	$\rightarrow 1$
1	$\rightarrow 2$
4	$\rightarrow 2$
3	$\rightarrow 3$
6	$\rightarrow 1$

* Code $\rightarrow$ {TODO}

03. Count of distinct ele.

$A[] = \{ 2, 3, 1, 3, 4 \}$ $Ans = 4$

$A[] = \{ 3, 3, 3 \}$ $Ans = 1$

* Idea $\rightarrow$ Use hashset & insert arr elements inside it.

```
HashSet < Integer > set;
```

```
for (i=0; i<n; i++) {
```

```
    set.add (A[i]);
```

```
}
```

```
return set.size();
```

TC: $O(n)$

SC: $O(n)$

Q5. Given $arr[N]$ elements, check if there exists a subarray with sum = 0

$arr[10] =$

2	2	1	-3	4	3	1	-2	-3	2
0	1	2	3	4	5	6	7	8	9

Ans = True

Brute force → Consider every subarray, & check if sum == 0 or not.

3 nested loops

TC: $O(n^3)$

SC: $O(1)$

Prefix sum

TC: $O(n^2)$

SC: $O(n)$

Carry forward

TC: $O(n^2)$

SC: $O(1)$

* Optimisation using prefix sum

$arr[10] =$

2	2	1	-3	4	3	1	-2	-3	2
0	1	2	3	4	5	6	7	8	9

$pf[10] =$

2	4	5	2	6	9	10	8	5	7
---	---	---	---	---	---	----	---	---	---

$$\text{sum}[i:j] = \text{psum}[j] - \text{psum}[i-1] = 0$$

$$\text{psum}[j] = \text{psum}[i-1]$$

$$\text{pf}[2] = 5 = \text{sum}[0-2]$$

$$\text{pf}[8] = 5 = \text{sum}[0-8]$$

$$\begin{aligned}\text{sum}[3-8] &= \text{pf}[8] - \text{pf}[2] \\ &= 5 - 5 \\ &= 0\end{aligned}$$

Conclusion $\rightarrow$ If we have a value repeating in our psum array, then we have a subarr with $\text{sum} = 0$

$$A[] = \{ 2 \ -5 \ 3 \ 6 \}$$

$$\text{pf}[] = \{ 2 \ -3 \ 0 \ 6 \}$$

Note - If 0 is present in pfsum[], we have a subarr with $\text{sum} = 0$

1st way = if ($\text{pf}[i] == 0$) return true;

2nd way = Explicitly add a 0 by yourself.

boolean subarrzero (int[] arr)

int pf[n]; $\longrightarrow$ {TODO}

HashSet<I> set;

set.add(0);

for (i=0; i<n; i++){

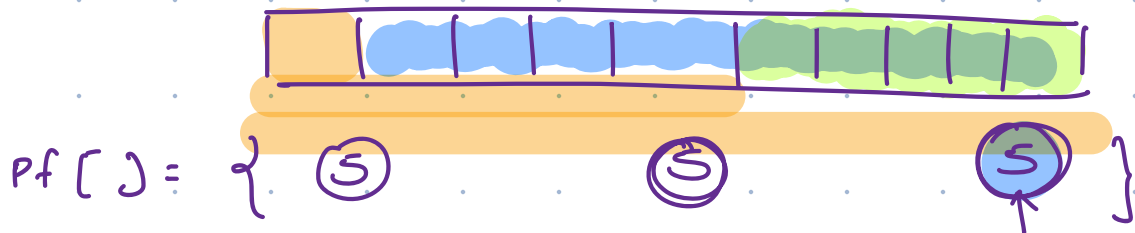
if (set.search(pf[i]) == True) return true;

set.add(pf[i]);

}

}

Count



5 $\rightarrow$ 1 2

ans = 0 + 1 + 2

$\rightarrow$ Use hashmap

$\rightarrow$ Edge case = map.insert(0, 1)

$\rightarrow$ ans = ans + hm.get(pf[i]);

